

Part 1

Code: Include pretty printed Verilog for assign2022 and MyPart1 modules.

```
module assign2022 (CLOCK_50, KEY, SW, LEDR, LEDG, HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7);
    input CLOCK_50;
    input [3:0] KEY;
    input [17:0] SW;
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7;
    output [17:0] LEDR;
    output [7:0] LEDG;

    assign LEDG[3:0] = ~KEY[3:0];
    assign LEDR[17:0] = SW[17:0];

    wire [19:0] student_id_decimal;

    MyPart1 MyPart1_inst(
        .iClk(CLOCK_50),
        .iRst(~KEY[0]),
        .iErrorCodes(SW[17:16]),
        .iEnableDisplay(SW[15]),
        .iFreezeDisplay(SW[14]),
        .oHEX0(HEX0),
        .oHEX1(HEX1),
        .oHEX2(HEX2),
        .oHEX3(HEX3));

    Bin2BCD Bin2BCD(.iBin14(14'd57), .oBCD20(student_id_decimal));
```

```
Hexdisplay H0(student_id_decimal[3:0], HEX6);  
Hexdisplay H1(student_id_decimal[7:4], HEX7);
```

endmodule

```
module MyPart1(iClk, iRst, iErrorCodes, iEnableDisplay, iFreezeDisplay, oHEX0, oHEX1, oHEX2, oHEX3);  
    input iClk;  
    input iRst;  
    input iEnableDisplay;  
    input iFreezeDisplay;  
    input [1:0] iErrorCodes;  
    output [6:0] oHEX0, oHEX1, oHEX2, oHEX3;  
    wire [15:0] oTime_100msec16;  
    wire reset_timer = (oTime_100msec16 == 16'd158);
```

```
Timer timer_inst(.iClk(iClk), .iRst(iRst), .iRstCE(reset_timer), .oTime_100msec16(oTime_100msec16));
```

```
DisplayTimerError DisplayTimerError_inst(  
    .iClk(iClk),  
    .iSWEntryError(iErrorCodes),  
    .iEnable(iEnableDisplay),  
    .iFreezeDisplayTimer(iFreezeDisplay),  
    .iTimer_100msec(oTime_100msec16),  
    .oHEX0(oHEX0),  
    .oHEX1(oHEX1),  
    .oHEX2(oHEX2),  
    .oHEX3(oHEX3));
```

endmodule

Question:

Explain how your design has made the time duration spent displaying your last count value 100 millisecond ?

Answer :

The DE2 board has of reference clock of 50 megahertz, every clock cycle increment by one. One second equals to 50 megahertz equals to 50 mega clock cycles. So 100 millisecond is 0.1 second which need 5 mega cycles.

Part 2

Code: Include pretty printed Verilog for assign2022 and NextIDDualOctal modules.

```
module assign2022(CLOCK_50, KEY, SW, LEDR, LEDG, HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7);  
    input CLOCK_50;  
    input [3:0] KEY;  
    input [17:0] SW;  
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7;  
    output [17:0] LEDR;  
    output [7:0] LEDG;  
  
    assign LEDG[3:0] = ~KEY[3:0];  
    assign LEDR[17:0] = SW[17:0];  
  
    // instantiate modules here for Parts 1, 2 and 3.  
    wire [19:0] student_id_decimal;  
    wire [5:0] IDdualOctal;  
    wire [3:0] hex4_muxed_display;  
    wire [3:0] hex5_muxed_display;  
    wire [3:0] hex4_random;  
    wire [3:0] hex5_random;  
    wire [11:0] LFSR12bit1;  
    wire [11:0] LFSR12bit2;  
  
    // simple debounce  
    reg debounced_key1;  
    always @(posedge CLOCK_50) begin  
        debounced_key1 <= KEY[1];  
    end  
endmodule
```

end

```
OctNumbersGenerationDisplay OctNumbersGenerationDisplay_inst(  
    .iClk(debounced_key1),  
    .iRst(~KEY[0]),  
    .iBlankDisplay(SW[15]),  
    .iNewOctNumsReq(SW[0]),  
    .iChooseRandID(SW[5]),  
    .oDualOctGenerated(),  
    .oHEXA(HEX4),  
    .oHEXB(HEX5)  
);
```

endmodule

```
module NextIDDualOctal(iClk, iRst, iNext, oIDDualOctal);  
    input iClk,           // System clock  
        iNext,           // Next value to be produced on 0->1 clock edge when ClockEnable=1  
        iRst;            // Resets to starting state.  
    output reg [5:0] oIDDualOctal; // The student ID value being produced  
  
    reg [3:0] my_id0;  
    reg [3:0] my_id1;  
    reg [3:0] my_id2;  
    reg [3:0] my_id3;  
    reg [3:0] my_id4;  
    reg [3:0] my_id5;  
    reg [3:0] my_id6;  
    reg [3:0] my_id7;
```

```
wire [6:0] id7_mul_11;
assign id7_mul_11 = my_id7 * 11;

always @(posedge iClk) begin
    if (iRst) begin
        oIDdualOctal    <= 6'd0;
        my_id0           <= 4'd7;
        my_id1           <= 4'd5;
        my_id2           <= 4'd8;
        my_id3           <= 4'd1;
        my_id4           <= 4'd7;
        my_id5           <= 4'd1;
        my_id6           <= 4'd0;
        my_id7           <= 4'd3;
    end
    else begin
        if (iNext) begin
            oIDdualOctal <= id7_mul_11[5:0];
            // rolling shift id register
            my_id0       <= my_id7;
            my_id1       <= my_id0;
            my_id2       <= my_id1;
            my_id3       <= my_id2;
            my_id4       <= my_id3;
            my_id5       <= my_id4;
            my_id6       <= my_id5;
            my_id7       <= my_id6;
        end
    end
end
```

```
        // even do nothing here, but equivalent to folloing code:
        // oIDdualOctal <= oIDdualOctal;
    end
end
end
endmodule
```

Question:

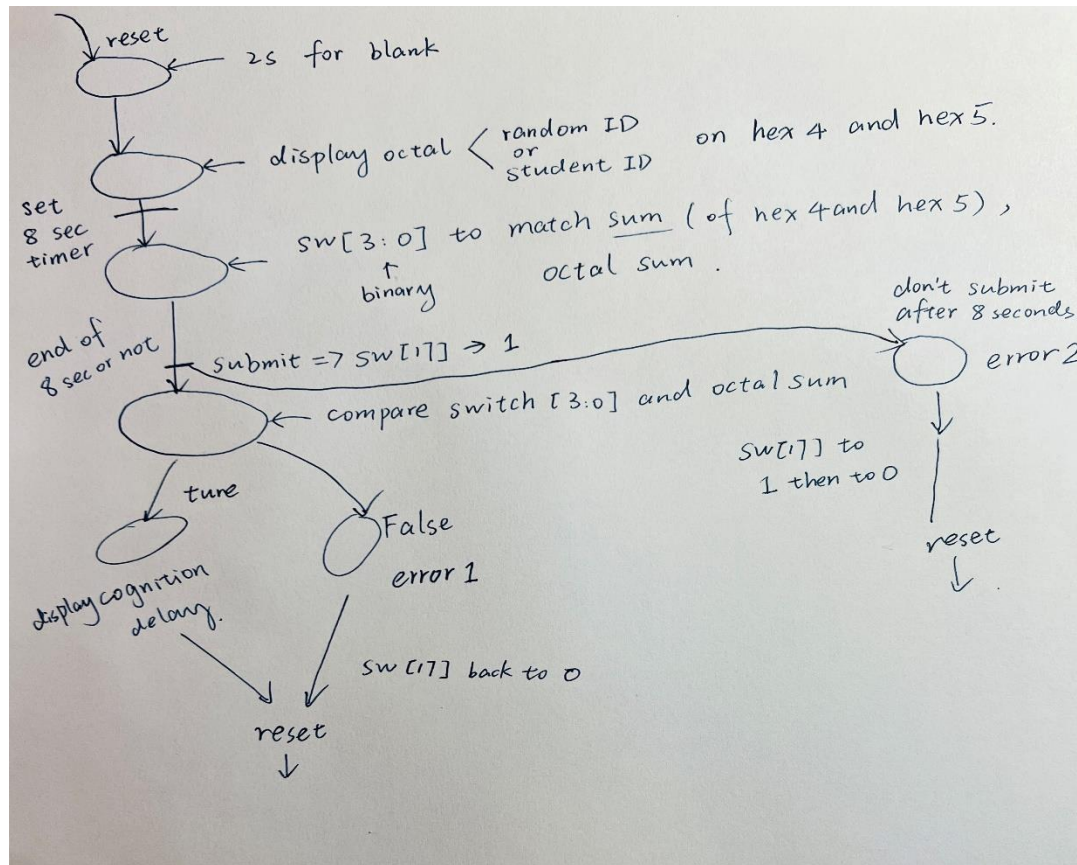
How many flip-flops should a minimal solution for module NextIDdualOctal generate? How many does yours generate?

Answer :

Since we have 8 digits student ID, we need 8×4 bits flip-flops to compose a rolling shift register. With output 6-bit width register oIDdualOctal, there are total $32 + 6 = 38$ bits. Thus, 38 flip-flops are needed.

Question:

How many states are there in a minimal state diagram for the FSM generated by a good solution for the NextIDDualOctal module?



Answer :

The diagram on the left is the FSM design state diagram. Because FSM uses NextIDDualOctal module, we can see there are 8 states as a good solution.

Part 3

Code: Include pretty printed Verilog ONLY for assign2022 and FSM modules.

```
module assign2022(CLOCK_50, KEY, SW, LEDR, LEDG, HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7);
    input CLOCK_50;
    input [3:0] KEY;
    input [17:0] SW;
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7;
    output [17:0] LEDR;
    output [7:0] LEDG;

    assign LEDG[3:0] = ~KEY[3:0];
    assign LEDR[17:0] = SW[17:0];

    // instantiate modules here for Parts 1, 2 and 3.
    wire [19:0] student_id_decimal;
    wire [5:0] IDdualOctal;
    wire [3:0] hex4_muxed_display;
    wire [3:0] hex5_muxed_display;
    wire [3:0] hex4_random;
    wire [3:0] hex5_random;
    wire [11:0] LFSR12bit1;
    wire [11:0] LFSR12bit2;

    CognitionTimer CognitionTimer_inst(
        .iClk          (CLOCK_50),
        .iRst          (~KEY[0]),
        .iChooseRandID (SW[16]),
```

```

.iUser4bitSW      (SW[3:0]),
.iSubmitSW        (SW[17]),
.oHEX0            (HEX0),
.oHEX1            (HEX1),
.oHEX2            (HEX2),
.oHEX3            (HEX3),
.oHEX4            (HEX4),
.oHEX5            (HEX5),
.oHEX6            (HEX6),
.oHEX7            (HEX7));
endmodule

```

```

module FSM(iClk, iRst, iUser4bitSW, iSubmitSW, iTimer16, iDualOctGenerated, oResetTimer, oUser4bitNumError,
           oFreezeDisplayTimer, oBlankOctNumsDisplay, oShowTimerErrorDisplay, oNewOctNumsReq, oState);

input iClk, iRst, iSubmitSW;           // iSubmitSW is the user operated submit switch
input [3:0] iUser4bitSW;               // 4 bit number on switches from the user
input [15:0] iTimer16;                 // time since oResetTimer becoming 0 in milliseconds
input [5:0] iDualOctGenerated;         // 4 bit number expected from user
output reg oResetTimer,                // zero timer value.
           oFreezeDisplayTimer,        // capture timer value for display if oShowTimerErrorDisplay=1.
           oBlankOctNumsDisplay,       // blank the octal numbers display.
           oNewOctNumsReq,             // a new 4 bit number will be generated on the next clock edge
           oShowTimerErrorDisplay;     // Show the captured time or error
output reg [1:0] oUser4bitNumError;    // 0 for no error, 1 wrong switch pressed, 2 timeout reached with no switches
1.
output [3:0] oState;                   // state available only for debugging.

```

```

reg [3:0] state, next_state;

assign oState = state;

// define states here
parameter RESET_STATE = 0;
parameter ENABLE_OCTAL_DISPLAY = 1;
parameter EIGHT_SEC_ERROR = 2;
parameter SUBMITTED_STATE = 3;
parameter COGNITION_DELAY = 4;
parameter ERROR1 = 5;
parameter EIGHT_SEC_ERROR_SW_SEQ = 6;
parameter WAIT_FOR_COGNITION_RST = 7;

reg enable_eight_sec_timer;
reg iUser4bitSW_d;
wire [6:0] octal_sum;
wire eight_sec_flag; //set as 1 if 8 sec
assign eight_sec_flag = (iTimer16 == 16'd100); // 80 + 20 = 10 sec

assign octal_sum = iDualOctGenerated[5:3] + iDualOctGenerated[2:0];

always@(state, iTimer16, iUser4bitSW, iDualOctGenerated, iSubmitSW) begin
    //default control actions are no action, for exxample:
    oFreezeDisplayTimer = 0;
    oBlankOctNumsDisplay = 0;
    oNewOctNumsReq = 0;
    enable_eight_sec_timer = 0;

```

```
oResetTimer = 0;
oUser4bitNumError = 2'b0;
oShowTimerErrorDisplay = 0;
```

```
case(state)
```

```
  RESET_STATE: begin
```

```
    // blank display for 2 sec
```

```
    if (iTimer16 == 16'd20) begin
```

```
      next_state = ENABLE_OCTAL_DISPLAY;
```

```
    end
```

```
    else begin
```

```
      next_state = RESET_STATE;
```

```
    end
```

```
  end
```

```
  // 1. enable display and request a group of two octal number
```

```
  // 2. start a eight sec timer
```

```
  ENABLE_OCTAL_DISPLAY: begin
```

```
    oBlankOctNumsDisplay = 1;
```

```
    oNewOctNumsReq = 1;
```

```
    enable_eight_sec_timer = 1;
```

```
    if (eight_sec_flag) begin
```

```
      next_state = EIGHT_SEC_ERROR;
```

```
    end
```

```
    else if (iSubmitSW) begin
```

```
      next_state = SUBMITTED_STATE;
```

```
    end
```

```
    else begin
```

```
      next_state = ENABLE_OCTAL_DISPLAY;
```

```

    end
end

EIGHT_SEC_ERROR: begin
    // wait iSubmitSW become 1
    oUser4bitNumError = 2'b10;
    oShowTimerErrorDisplay = 1;
    if (iSubmitSW) begin
        next_state = EIGHT_SEC_ERROR_SW_SEQ;
    end
    else begin
        next_state = EIGHT_SEC_ERROR;
    end
end

EIGHT_SEC_ERROR_SW_SEQ: begin
    // wait iSubmitSW become 0
    oUser4bitNumError = 2'b10;
    oShowTimerErrorDisplay = 1;
    if (iSubmitSW == 1'b0) begin
        next_state = RESET_STATE;
        oResetTimer = 1;
    end
    else begin
        next_state = EIGHT_SEC_ERROR_SW_SEQ;
    end
end

SUBMITTED_STATE: begin

```

```

        // go to cognition delay and reset timer
        if (octal_sum[3:0] == iUser4bitSW) begin
            next_state = COGNITION_DELAY;
            oResetTimer = 1;
        end
        else begin
            next_state = ERROR1;
        end
    end
end

COGNITION_DELAY: begin
    if (iTimer16 == iUser4bitSW) begin
        next_state = WAIT_FOR_COGNITION_RST;
    end
    else begin
        next_state = COGNITION_DELAY;
    end
end

WAIT_FOR_COGNITION_RST: begin
    oFreezeDisplayTimer = 1;
    if (iSubmitSW == 1'b0) begin
        next_state = RESET_STATE;
        oResetTimer = 1;
    end
end

ERROR1: begin
    oShowTimerErrorDisplay = 1;
    oUser4bitNumError = 2'b01;
end

```

```
        if (iSubmitSW == 1'b0) begin
            next_state = RESET_STATE;
            oResetTimer = 1;
        end
    end
end
```

```
default: begin
    next_state = RESET_STATE;
end
```

```
endcase
end
```

```
always @(posedge iClk) begin // Update state on iClk edge
    if (iRst) state <= RESET_STATE;
    else state <= next_state;
end
```

```
always @(posedge iClk ) begin // debounce logic
    iUser4bitSW_d <= iUser4bitSW;
end
```

```
endmodule
```